

0.65.5. La fonction bouton() — lire la manette

#include <bouton> : bouton(A) vaut 1 tant que le bouton A est enfoncé, 0 sinon.

1. Ce qu'il faut comprendre

Ce qu'elle fait : bouton(X) rend 1 tant que le bouton est **enfoncé**, 0 sinon. Les huit : A, B, START, SELECT, HAUT, BAS, GAUCHE, DROITE.

La ligne à écrire : #include <bouton>.

Son argument : le nom du bouton.

Ce qu'elle coûte : environ **50 octets** : la routine qui lit les huit boutons, et un test par appel.

Essaie : un deuxième if, avec bouton(B), qui écrit ailleurs.

On retrouve bouton dans 137 leçons

[0.66](#), [0.66.1](#), [0.66.2](#), [0.67](#), [0.67.1](#), [0.67.2](#), [0.68](#), [0.68.1](#), [0.68.2](#), [0.69](#), [0.69.1](#), [0.69.2](#), [0.69.3](#), [0.70](#), [0.70.1](#), [0.70.2](#), [0.82](#), [0.82.1](#), [0.83.2](#), [0.84](#), [0.90.1](#), [0.90.2](#), [0.90.4](#), [0.90.5](#), [0.90.6](#), [0.90.7](#), [0.90.8](#), [0.91](#), [0.91.1](#), [0.91.2](#), [0.91.3](#), [0.91.4](#), [0.91.5](#), [0.91.6](#), [0.91.7](#), [0.91.8](#), [0.91.9](#), [0.92](#), [0.92.1](#), [0.92.2](#), [0.93.1](#), [0.93.2](#), [0.93.3](#), [0.93.4](#), [0.93.5](#), [0.93.6](#), [0.93.7](#), [0.93.8](#), [0.93.9](#), [0.94](#), [0.94.1](#), [0.95](#), [0.95.1](#), [0.97](#), [0.97.1](#), [0.98](#), [0.98.1](#), [0.99](#), [0.99.1](#), [0.100](#), [0.100.1](#), [0.101](#), [0.101.1](#), [0.102](#), [0.102.1](#), [0.103](#), [0.104](#), [0.105](#), [0.105.1](#), [0.106](#), [0.107](#), [0.107.1](#), [0.108](#), [2](#), [2.1](#), [2.2](#), [2.3](#), [14](#), [16](#), [17](#), [18](#), [19](#), [20](#), [21](#), [23](#), [44](#), [45](#), [47](#), [48](#), [49](#), [51](#), [52](#), [53](#), [54](#), [55](#), [57](#), [70](#), [81](#), [82](#), [84](#), [85](#), [89](#), [91](#), [94](#), [97](#), [98](#), [100](#), [102](#), [106](#), [107](#), [108](#), [109](#), [110](#), [111](#), [112](#), [113](#), [114](#), [116](#), [117](#), [118](#), [119](#), [120](#), [121](#), [122](#), [123](#), [124](#), [125](#), [126](#), [127](#), [129](#), [131](#), [132](#), [134](#), [136](#), [137](#), [138](#), [143](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **465 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         48 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireTexte — attend de pouvoir écrire une case
EffacerCases   12 o ← effacer() — efface des cases
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
CopierTuiles   85 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
LireManette    37 o ← bouton() — lit les huit boutons
Tuiles         128 o ← CopierTuiles — 8 lettres sur 43
"APPUIE SUR A" 12 o ← texte() — un texte gravé
"APPUYE"        6 o ← texte() — un texte gravé
```

Lettres gravées : AEIPRSUY

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,

DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 2	22 o sur place, dans main()	+ EcrireTexte (10 o)
effacer()	× 1	8 o sur place, dans main()	+ EffacerCases (12 o)
bouton()	× 1	5 o sur place, dans main()	+ LireManette (37 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

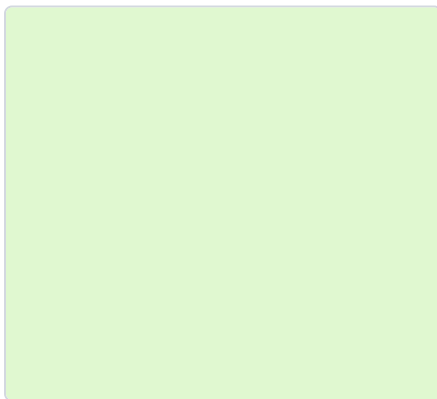
```
// ---- #include <bouton> : lire la manette ----  
// Tant que A est enfoncé, APPUYE s'affiche ; relâché, il s'efface.  
  
#include <bouton>    // bouton() : 1 si le bouton est enfoncé  
#include <texte>  
#include <effacer>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

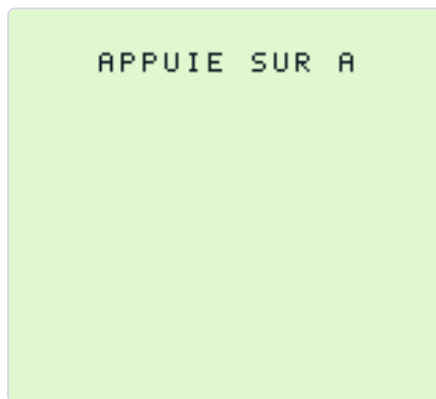
```
int main() {  
    texte(4, 2, "APPUIE SUR A");    // la consigne, en haut  
  
    while (true) {  
        image();  
        if (bouton(A)) {  
            texte(6, 8, "APPUYE");  
        } else {  
            effacer(6, 8, "APPUYE");  
        }  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

APPUIE SUR A

A — l'écran n'a pas bougé

5. Ce qu'on doit voir

APPUIE SUR A en haut ; APPUYE apparaît au milieu tant qu'on tient le bouton A.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ rien au départ

✓ A tenu : APPUYE

✓ relâché : effacé

Ensuite : **0.66. La lettre bouge avec la croix** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++